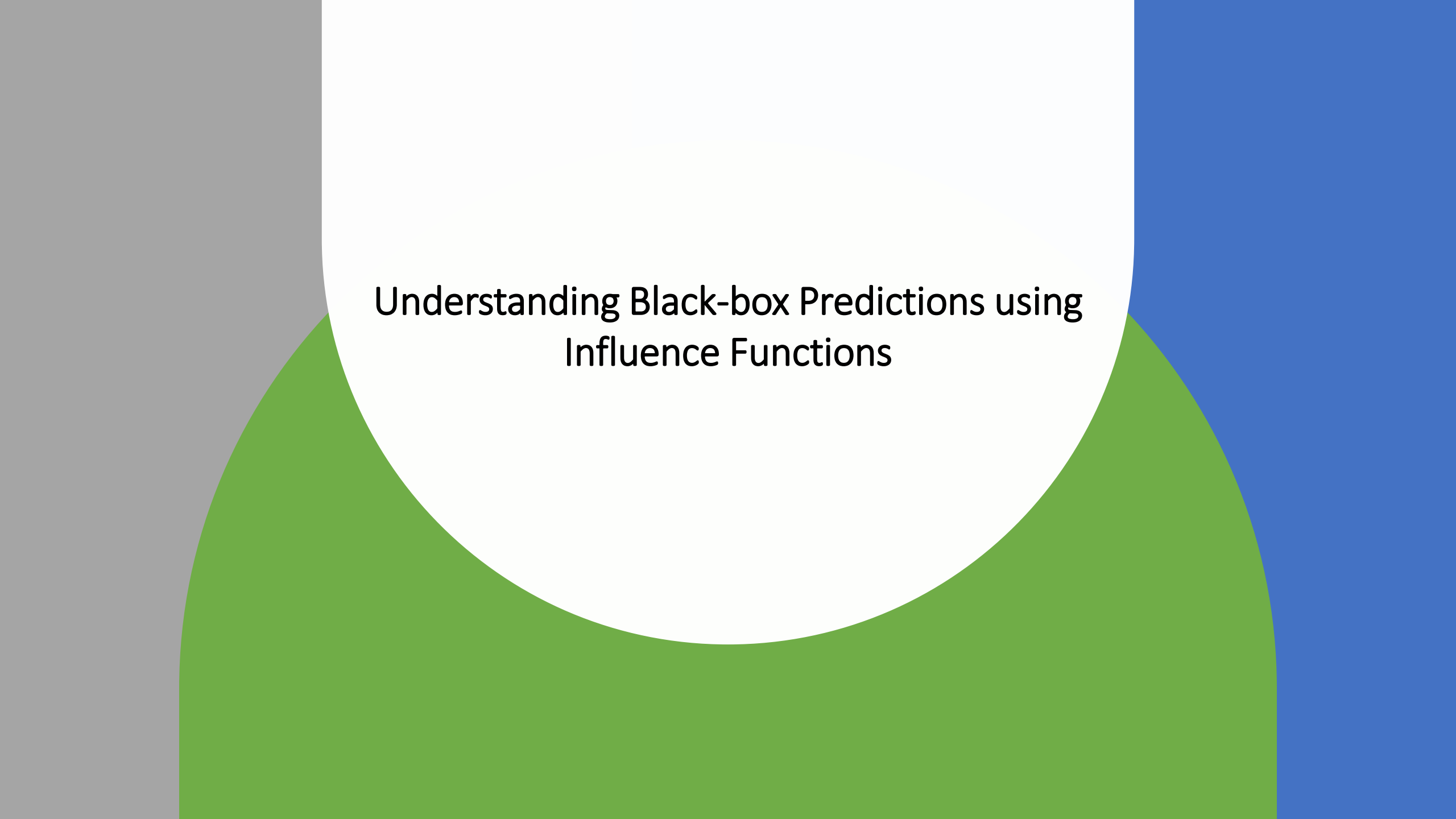




Understanding Black-box Predictions using Influence Functions

Agenda

Introduction

Approach

Calculation

Comparison

Uses



Introduction

- How to understand predictions of a black-box?
- Perturbation or local linear fit
- Trace back to training data and perturb.
- Expensive! Use Influence Functions.





Influence Functions

Approach

Input space $\mathcal{X}$ and output space $\mathcal{Y}$.

Training points $z_1, \dots, z_n; z_i = (x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$

$L(z, \theta)$ is Loss and Empirical Risk:- $\frac{1}{n} \sum_{i=1}^n L(z_i, \theta)$

Minimizer:- $\hat{\theta} \stackrel{\text{def}}{=} \operatorname{argmin}_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n L(z_i, \theta)$

Assumption- twice differentiable and convex



Upweighting Training point

Change is $\hat{\theta}_{-z} - \hat{\theta}$ but it is slow

where $\hat{\theta} \stackrel{\text{def}}{=} \operatorname{argmin}_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n L(z_i, \theta)$

Instead we update parameter after changing z by ϵ

$$\hat{\theta}_{\epsilon, z} \stackrel{\text{def}}{=} \operatorname{argmin}_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n L(z_i, \theta) + \epsilon L(z, \theta)$$

Influence:- $\mathcal{I}_{\text{up, params}}(z) \stackrel{\text{def}}{=} \left. \frac{d\hat{\theta}_{\epsilon, z}}{d\epsilon} \right|_{\epsilon=0} = -H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}),$

where $H_{\hat{\theta}} \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i=1}^n \nabla_{\theta}^2 L(z_i, \hat{\theta})$ is the Hessian

Removing means $\epsilon = -\frac{1}{n}$ then $\hat{\theta}_{-z} - \hat{\theta} \approx -\frac{1}{n} \mathcal{I}_{\text{up, params}}(z)$

$$\mathcal{I}_{\text{up, loss}}(z, z_{\text{test}}) \stackrel{\text{def}}{=} \left. \frac{dL(z_{\text{test}}, \hat{\theta}_{\epsilon, z})}{d\epsilon} \right|_{\epsilon=0}$$

Chain Rule :-

$$\begin{aligned} &= \nabla_{\theta} L(z_{\text{test}}, \hat{\theta})^{\top} \left. \frac{d\hat{\theta}_{\epsilon, z}}{d\epsilon} \right|_{\epsilon=0} \\ &= -\nabla_{\theta} L(z_{\text{test}}, \hat{\theta})^{\top} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}). \end{aligned}$$

Perturbing training point

Perturb $z \mapsto z_\delta$ where $z = (x, y)$ and $z_\delta \stackrel{\text{def}}{=} (x + \delta, y)$

Risk min $\hat{\theta}_{\epsilon, z_\delta, -z} \stackrel{\text{def}}{=} \operatorname{argmin}_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n L(z_i, \theta) + \epsilon L(z_\delta, \theta) - \epsilon L(z, \theta)$

$$\begin{aligned} \left. \frac{d\hat{\theta}_{\epsilon, z_\delta, -z}}{d\epsilon} \right|_{\epsilon=0} &= \mathcal{I}_{\text{up, params}}(z_\delta) - \mathcal{I}_{\text{up, params}}(z) \\ &= -H_{\hat{\theta}}^{-1}(\nabla_{\theta} L(z_\delta, \hat{\theta}) - \nabla_{\theta} L(z, \hat{\theta})). \end{aligned}$$

Same linear approx. $\hat{\theta}_{z_\delta, -z} - \hat{\theta} \approx -\frac{1}{n} \left(\mathcal{I}_{\text{up, params}}(z_\delta) - \mathcal{I}_{\text{up, params}}(z) \right)$

Similar for y

Combine both for discrete data.

x is continuous and δ is small

Input domain $\mathcal{X} \subseteq \mathbb{R}^d$ and parameters $\Theta \subseteq \mathbb{R}^p$ and L is differentiable in θ and x

As $||\delta|| \rightarrow 0$, $\nabla_{\theta} L(z_{\delta}, \hat{\theta}) - \nabla_{\theta} L(z, \hat{\theta}) \approx [\nabla_x \nabla_{\theta} L(z, \hat{\theta})] \delta$ where $[\nabla_x \nabla_{\theta} L(z, \hat{\theta})] \delta \in \mathbb{R}^{p \times d}$

Substituting, $\left. \frac{d\theta_{\epsilon, z_{\delta}, -z}}{d\epsilon} \right|_{\epsilon=0} \approx -H_{\hat{\theta}}^{-1} [\nabla_x \nabla_{\theta} L(z, \hat{\theta})] \delta.$

We have, $\hat{\theta}_{z_{\delta}, -z} - \hat{\theta} \approx -\frac{1}{n} H_{\hat{\theta}}^{-1} [\nabla_x \nabla_{\theta} L(z, \hat{\theta})] \delta.$

Differentiating w.r.t. δ and applying chain rule we get

$$\begin{aligned} \mathcal{I}_{\text{pert,loss}}(z, z_{\text{test}}) &\stackrel{\text{def}}{=} \nabla_{\delta} L(z_{\text{test}}, \hat{\theta}_{z_{\delta}, -z}) \Big|_{\delta=0} \\ &= -\nabla_{\theta} L(z_{\text{test}}, \hat{\theta})^{\top} H_{\hat{\theta}}^{-1} \nabla_x \nabla_{\theta} L(z, \hat{\theta}). \end{aligned} \tag{5}$$

This tells us the approximate effect that $z \mapsto z + \delta$ has on loss at z_{test}

We identify the features of z that are most responsible for the prediction on z_{test}

Relation to Euclidean distance

Maximize $x \cdot x_{\text{test}}$

Let $p(y|x) = \sigma(y\theta^T x)$, with $y \in \{-1, 1\}$ and σ is sigmoid

For a training point $z = (x, y)$, $L(z, \theta) = \log(1 + \exp(-y\theta^T x))$,

$$\nabla_{\theta} L(z, \theta) = -\sigma(-y\theta^T x)yx \text{ and } H_{\theta} = \frac{1}{n} \sum_{i=1}^n \sigma'(\theta^T x_i) \sigma'(-\theta^T x_i) x_i x_i^T.$$

$\mathcal{I}_{\text{up,loss}}(z, z_{\text{test}})$ is:

$$-y_{\text{test}}y \cdot \sigma(-y_{\text{test}}\theta^T x_{\text{test}}) \cdot \sigma(-y\theta^T x) \cdot x_{\text{test}}^T H_{\hat{\theta}}^{-1} x.$$

Two key differences:-

- High training loss has more influence
- Weighted covariance matrix measure “resistance”.

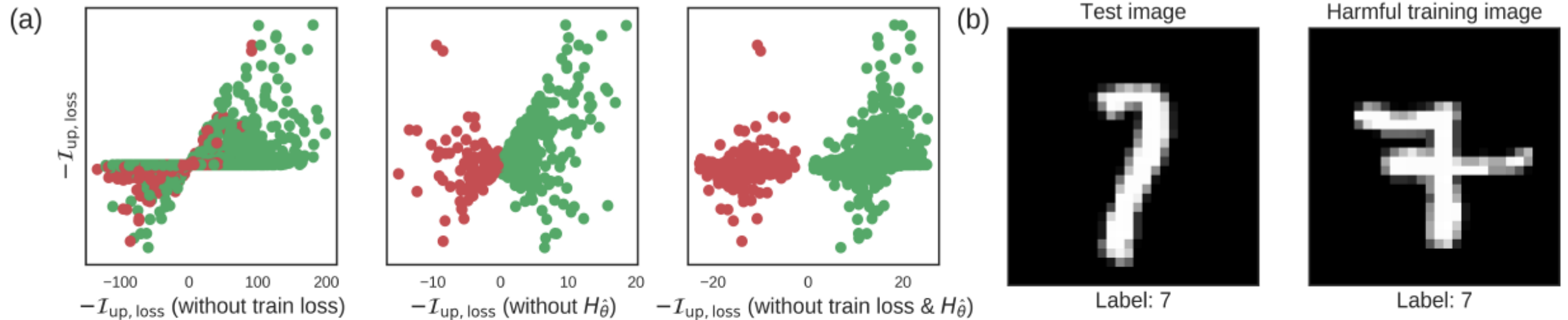


Figure 1. Components of influence. (a) What is the effect of the training loss and $H_{\hat{\theta}}^{-1}$ terms in $\mathcal{I}_{\text{up}, \text{loss}}$? Here, we plot $\mathcal{I}_{\text{up}, \text{loss}}$ against variants that are missing these terms and show that they are necessary for picking up the truly influential training points. For these calculations, we use logistic regression to distinguish 1’s from 7’s in MNIST (LeCun et al., 1998), picking an arbitrary test point z_{test} ; similar trends hold across other test points. Green dots are train images of the same label as the test image (7) while red dots are 1’s. **Left:** Without the train loss term, we overestimate the influence of many training points: the points near the horizontal axis should have $\mathcal{I}_{\text{up}, \text{loss}}$ close to 0, but instead have high influence when we remove the train loss term). **Mid:** Without $H_{\hat{\theta}}^{-1}$, all green training points are helpful (removing each point increases test loss) and all red points are harmful (removing each point decreases test loss). This is because $\forall x, x \succeq 0$ (all pixel values are positive), so $x \cdot x_{\text{test}} \geq 0$, but it is incorrect: many harmful training points actually share the same label as z_{test} . See panel (b). **Right:** Without training loss or $H_{\hat{\theta}}^{-1}$, what is left is the scaled Euclidean inner product $y_{\text{test}} y \cdot \sigma(-y_{\text{test}} \theta^{\top} \cdot x_{\text{test}}) x_{\text{test}}^{\top} x$, which fails to accurately capture influence; the scatter plot deviates quite far from the diagonal. (b) The test image and a harmful training image with the same label. To the model, they look very different, so the presence of the training image makes the model think that the test image is less likely to be a 7. The Euclidean inner product does not pick up on these less intuitive, but important, harmful influences.

Efficiently Calculating Influence

Inverting Hessian requires $\mathcal{O}(np^2 + p^3)$ and also need to calculate across all z_i

Use Hessian-vector products to approximate $s_{\text{test}} \stackrel{\text{def}}{=} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z_{\text{test}}, \hat{\theta})$

and then compute $\mathcal{I}_{\text{up,loss}}(z, z_{\text{test}}) = -s_{\text{test}} \cdot \nabla_{\theta} L(z, \hat{\theta})$

- Conjugate Gradients :- Hessian is pd!! $\mathcal{H}_{\hat{\theta}}^{-1}v \equiv \text{argmin}_t \{t^T \mathcal{H}_{\hat{\theta}} t - v^T t\} . \mathcal{O}(np)$
- Stochastic estimation:- Use Taylor Expansion! Sample t points from the training data.

Define $\hat{\mathcal{H}}_0^{-1}v = v$ and recursively compute $\hat{\mathcal{H}}_j^{-1}v = v + (I - \nabla_{\theta}^2 L(z_{s_j}, \hat{\theta})) \hat{\mathcal{H}}_{j-1}^{-1}v$

Repeat this process r times.

Original technique was only for GLM, a more general algorithm is used. $\mathcal{O}(np + rtp)$

Similarly, we compute $\mathcal{I}_{\text{pert,loss}}(z_i, z_{\text{test}}) = -\frac{1}{n} \nabla_{\theta} L(z_{\text{test}}, \hat{\theta})^T \hat{H}_{\hat{\theta}}^{-1} \nabla_x \nabla_{\theta} L(z_i, \hat{\theta})$

Using the HVP trick. This is easy to implement in Tensorflow.

Influence function vs leave-one out retraining

Asymptotic approximation. Parameters minimize risk (convex and twice differentiable)

Comparison done in MNIST data.

In practice, the Hessian is not pd. We use convex quadratic approximation of the loss

around $\tilde{\theta}$. $\tilde{L}(z, \theta) = L(z, \tilde{\theta}) + \nabla_{\theta} L(z, \tilde{\theta})^T (\theta - \tilde{\theta}) + \frac{1}{2} (H_{\tilde{\theta}} + \lambda I) (\theta - \tilde{\theta})$.

λ is damping term for negative eigenvalues.

For non-differentiable losses, influence functions computed on smooth approximations can predict the behaviour of original leave-one-out retraining.

Tested on a linear SVM on 1s vs 7s MNIST task



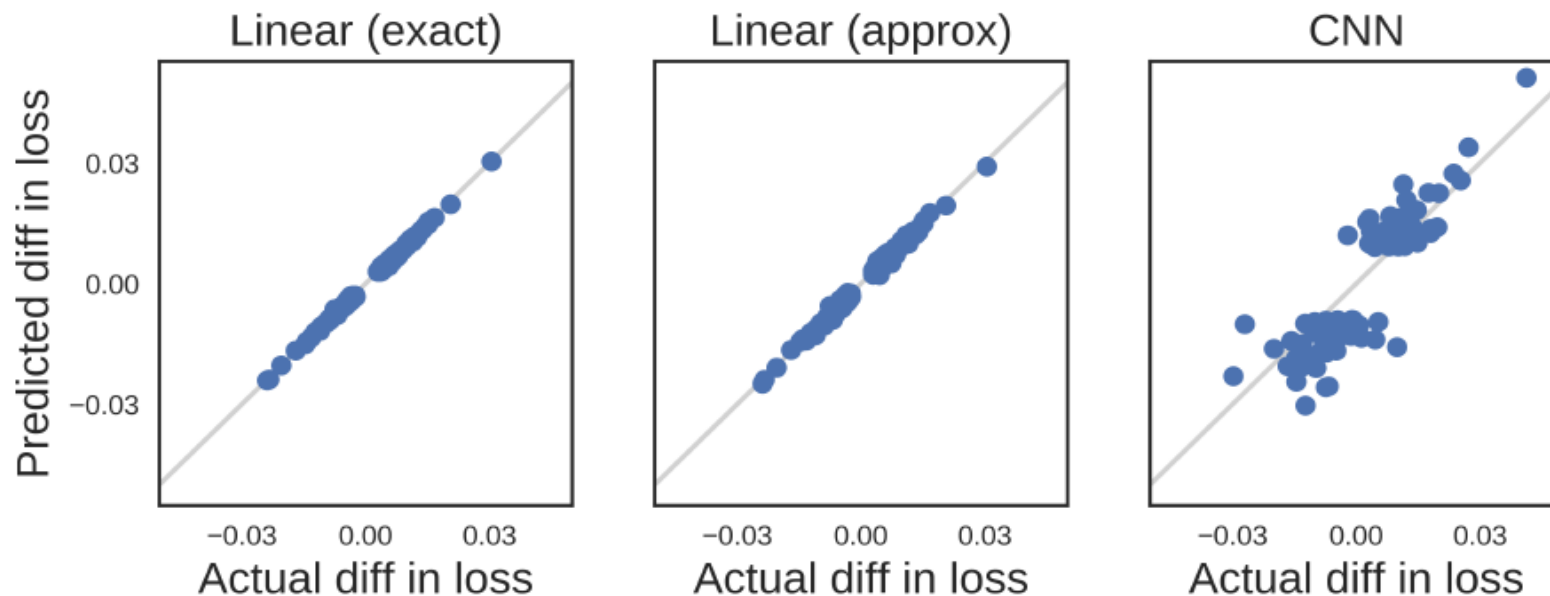


Figure 2. Influence matches leave-one-out retraining. We arbitrarily picked a wrongly-classified test point z_{test} , but this trend held more broadly. These results are from 10-class MNIST. **Left:** For each of the 500 training points z with largest $|\mathcal{I}_{\text{up,loss}}(z, z_{\text{test}})|$, we plotted $-\frac{1}{n} \cdot \mathcal{I}_{\text{up,loss}}(z, z_{\text{test}})$ against the actual change in test loss after removing that point and retraining. The inverse HVP was solved exactly with CG. **Mid:** Same, but with the stochastic approximation. **Right:** The same plot for a CNN, computed on the 100 most influential points with CG. For the actual difference in loss, we removed each point and retrained from $\tilde{\theta}$ for 30k steps.

Validation and Extension

Convex Quadratic approximation

λ damping term for negative eigenvalues

Derivative doesn't exist?

Example:- use smoothHinge instead of Hinge.



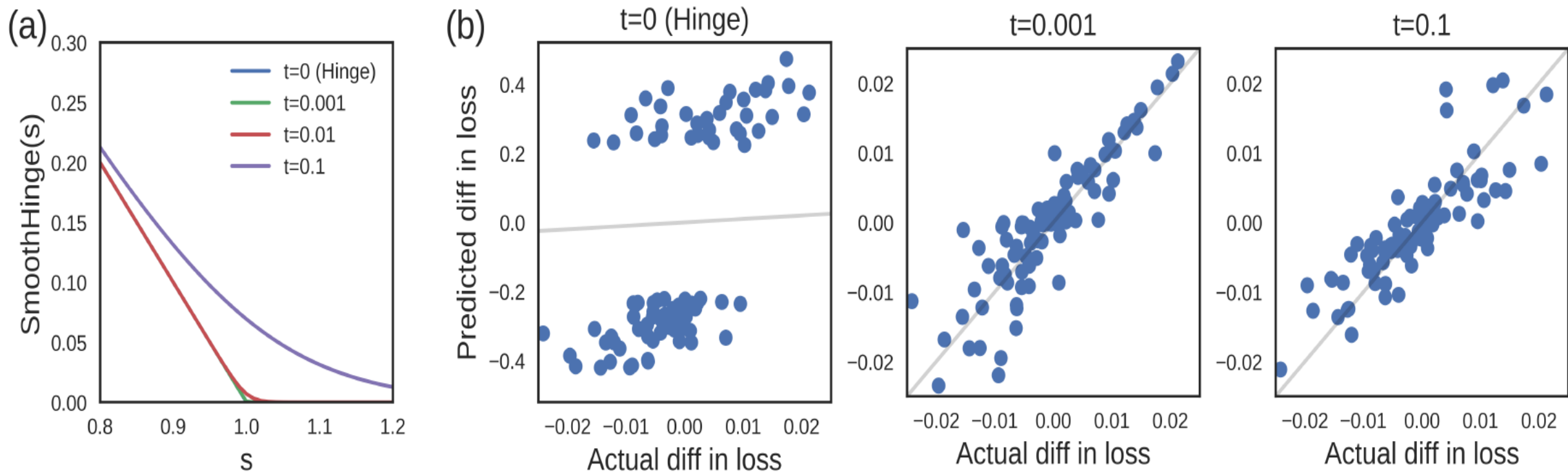
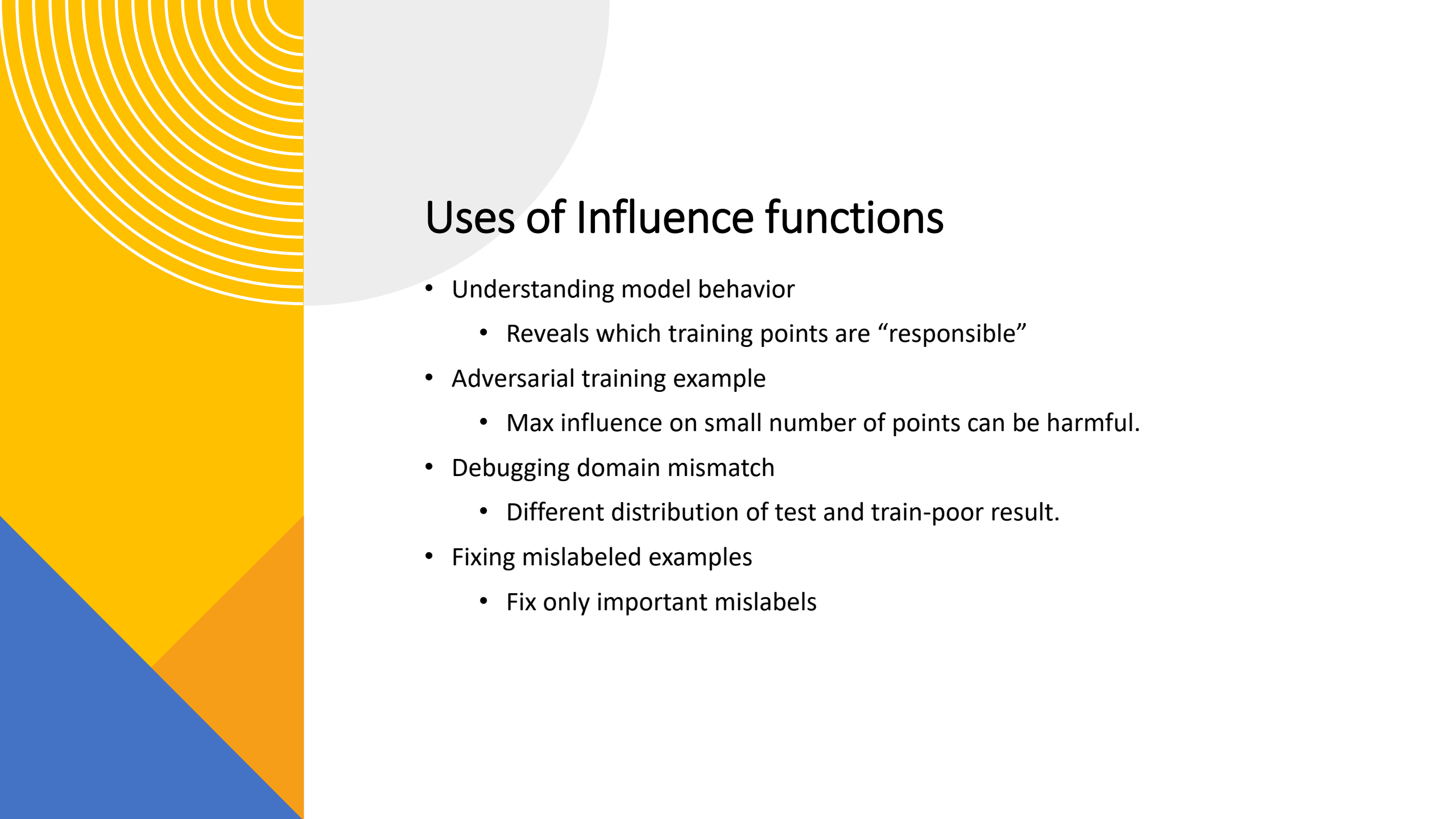


Figure 3. Smooth approximations to the hinge loss. (a) By varying t , we can approximate the hinge loss with arbitrary accuracy: the green and blue lines are overlaid on top of each other. (b) Using a random, wrongly-classified test point, we compared the predicted vs. actual differences in loss after leave-one-out retraining on the 100 most influential training points. A similar trend held for other test points. The SVM objective is to minimize $0.005 \|w\|_2^2 + \frac{1}{n} \sum_i \text{Hinge}(y_i w^\top x_i)$. **Left:** Influence functions were unable to accurately predict the change, overestimating its magnitude considerably. **Mid:** Using $\text{SmoothHinge}(\cdot, 0.001)$ let us accurately predict the change in the hinge loss after retraining. **Right:** Correlation remained high over a wide range of t , though it degrades when t is too large. When $t = 0.001$, Pearson's $R = 0.95$; when $t = 0.1$, Pearson's $R = 0.91$.



Uses of Influence functions

- Understanding model behavior
 - Reveals which training points are “responsible”
- Adversarial training example
 - Max influence on small number of points can be harmful.
- Debugging domain mismatch
 - Different distribution of test and train-poor result.
- Fixing mislabeled examples
 - Fix only important mislabels

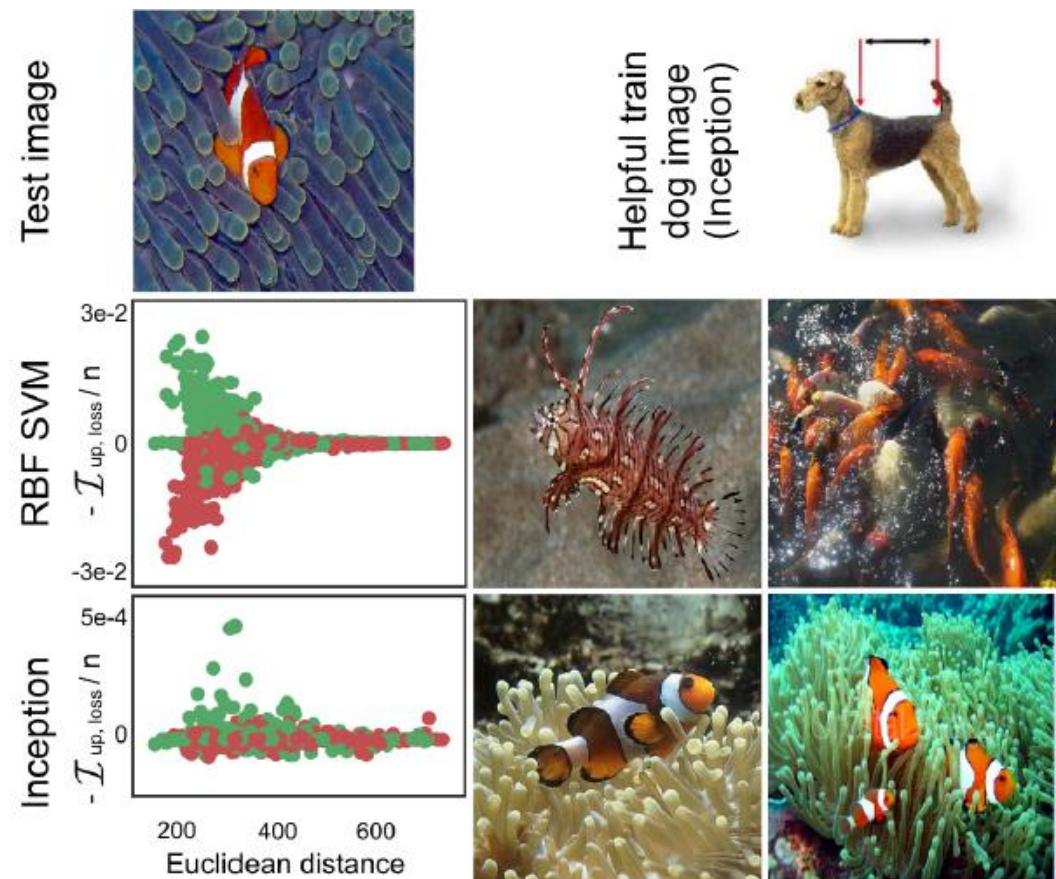


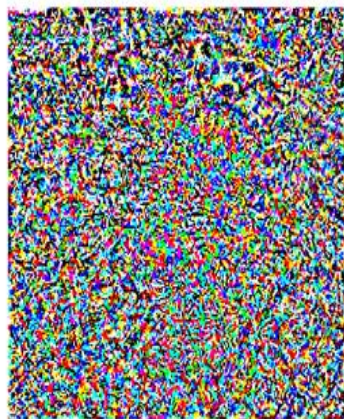
Figure 4. Inception vs. RBF SVM. Bottom left: $-\mathcal{I}_{\text{up,loss}}(z, z_{\text{test}})$ vs. $\|z - z_{\text{test}}\|_2^2$. Green dots are fish and red dots are dogs. Bottom right: The two most helpful training images, for each model, on the test. Top right: An image of a dog in the training set that helped the Inception model correctly classify the test image as a fish.

A small
perturbation
to one
training
example:

Label: Fish



+ ϵ ·



Label: Fish



Can change
multiple **test**
predictions:



Orig (confidence): Dog (97%)

Dog (98%)

Dog (98%)

Dog (99%)

Dog (98%)

New (confidence): Fish (97%)

Fish (93%)

Fish (87%)

Fish (60%)

Fish (51%)

Figure 5. Training-set attacks. We targeted a set of 30 test images featuring the first author's dog in a variety of poses and backgrounds. By maximizing the average loss over these 30 images, we found a visually-imperceptible change to the particular training image (shown on top) that flipped predictions on 16 test images.

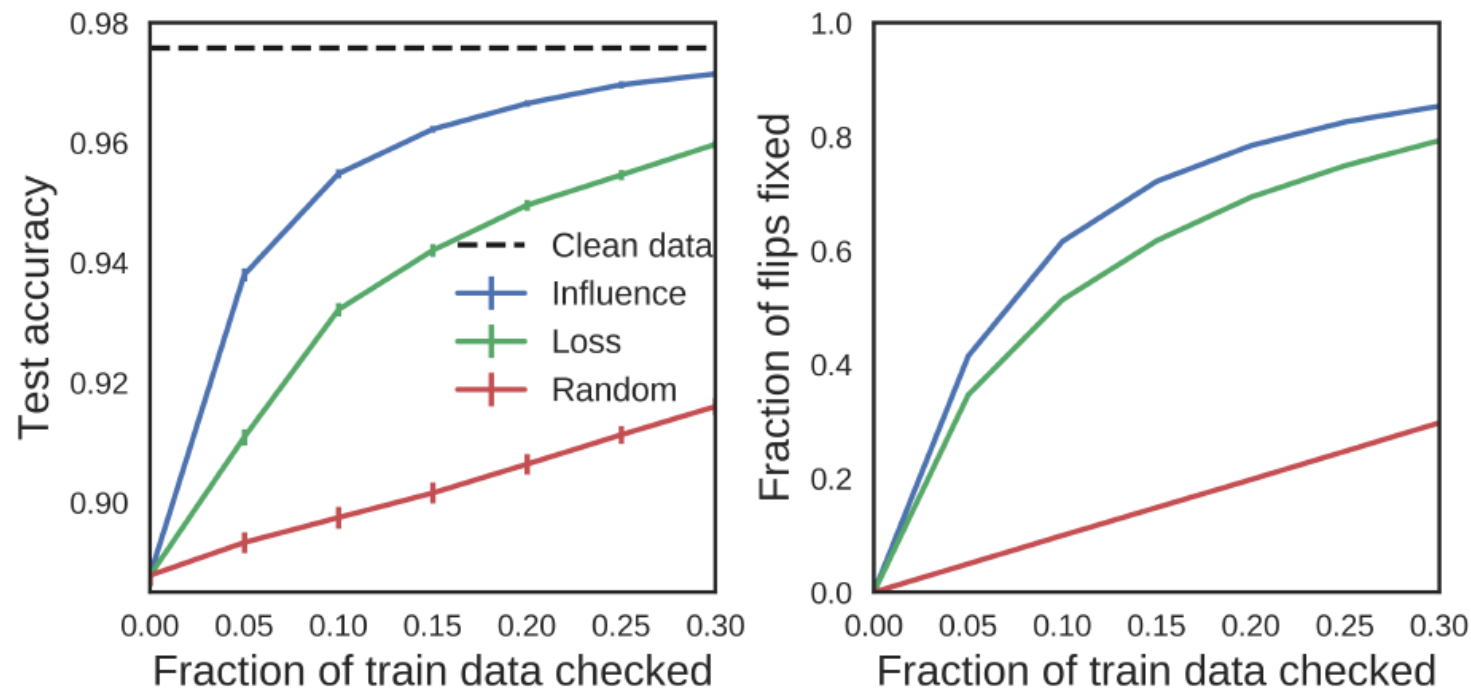


Figure 6. Fixing mislabeled examples. Plots of how test accuracy (left) and the fraction of flipped data detected (right) change with the fraction of train data checked, using different algorithms for picking points to check. Error bars show the std. dev. across 40 repeats of this experiment, with a different subset of labels flipped in each; error bars on the right are too small to be seen. These results are on the Enron1 spam dataset (Metsis et al., 2006), with 4,147 training and 1,035 test examples; we trained logistic regression on a bag-of-words representation of the emails.

Thank
you

Sagnik Barman

